



RoAN: A relation-oriented attention network for temporal knowledge graph completion

Luyi Bai^{a,b,*}, Xiangnan Ma^a, Xiangxi Meng^a, Xin Ren^a, Yujing Ke^a

^a School of Computer and Communication Engineering, Northeastern University (Qinhuangdao), Qinhuangdao 066004, China

^b School of Informatics, University of Leicester, Leicester LE1 7RH, United Kingdom

ARTICLE INFO

Keywords:

Attention mechanism

Completion

Temporal knowledge graph

ABSTRACT

In the last few years, the availability of temporal knowledge graphs (TKGs), which associate time information for each event, increased the need for completing in these TKGs. In order to solve the completion problem, the existing efforts extend static knowledge graph completion models to handle time-dependent representations, or encode the structural information of events for passing temporal message. However, the above efforts mostly construct TKGs in the form of multi-edges mesh and focus on entity features. They do not explore the power of relations for TKG completion task. In this paper, we introduce a form of relational multi-chains to reconstruct TKGs and propose a relation-oriented attention mechanism for embedding the features of relations. According to our attention mechanism, we build a Relation-oriented Attention Network (RoAN) to model temporal embeddings of relations. It is worth noting that our approach is model-agnostic and can be potentially combined with most existing TKG completion models. Experimental results show that our approach can be coupled with previous TKGC methods and can increase their performance accordingly. In addition, the analysis reveals the principle of our relation-oriented attention mechanism.

1. Introduction

Temporal Knowledge graphs (TKGs) have been applied to a wide variety of applications, such as recommendation systems (Cao et al., 2019), question answering (Do and Phan, 2022), social network analysis (Nguyen et al., 2018), and event forecasting (De Winter et al., 2018). Different from the triple (s, r, o) representation of event in static knowledge graphs, TKGs additionally associates each event with a timestamp and represents event as a quadruple (s, r, o, t) . GDEL (Leetaru and Schrodt, 2013) and ICEWS (Boschee et al., 2015) are two typical TKGs to describe who did what to whom, and when. Usually, TKGs can be represented as a sequence of static KG snapshots because they are assumed to consist of discrete timestamps as described in Jiang et al. (2016).

Due to the incompleteness of existing TKGs, predicting missing events in these temporal knowledge graphs has become an essential task, which is defined as temporal knowledge graph completion (TKGC). Given a temporal knowledge graph with timestamps varying from t_0 to t_T , TKGC primarily has two settings: *interpolation* and *extrapolation*. In the interpolation setting, new facts are predicted for time t such that $t_0 \leq t \leq t_T$. It can use both historical ($\leq t$) and future ($\geq t$) information. In contrast, *extrapolation* setting focuses on predicting new facts over timestamps t that are greater than t_T ($t > t_T$). It can only use historical information ($\leq t$). Therefore, *interpolation* often has higher accuracy and practical application value than *extrapolation*. At the same

time, *interpolation* is also the research task of this paper. Most work on TKGC have largely focused on extending current static KG completion models. They develop the time-dependent scoring functions to calculate the probability of missing events (e.g., TTransE (Leblay and Chekol, 2018), HyTe (Dasgupta et al., 2018), TA models (García-Durán et al., 2018), and DE models (Goel et al., 2020)). In addition, some tensor-based models have been proposed, such as ConT (Ma et al., 2019) and ATiSE (Xu et al., 2020).

In recent years, some attention-based networks such as GAT (Veličković et al., 2017) and RGAT (Busbridge et al., 2019) provide the framework to model structural information of temporal knowledge graphs. They mainly construct TKGs in the form of multi-edges mesh as shown in Fig. 1(a). There are multiple edges with different relations and timestamps between two entities in an entity pair (e.g. (United States $\rightarrow$ Consult $\rightarrow$ United Kingdom, 2018-11-15) and (United States $\rightarrow$ Cooperation $\rightarrow$ United Kingdom, 2018-11-22)). However, such form may be inappropriate for TKGC task, because the relations between two entities are dependent, existing attention-based networks pay more attention to obtaining the representation vector of entities, and they often use the original representation vector of relations. In order to solve this problem, we introduce a form of relational multi-chains to reconstruct temporal knowledge graphs as shown in Fig. 1(b). The arrowed line represents the timeline, and the solid line represents the relation between the two entities at that time. The relations of each entity

* Corresponding author at: School of Computer and Communication Engineering, Northeastern University (Qinhuangdao), Qinhuangdao 066004, China.
E-mail address: baily@neuq.edu.cn (L. Bai).

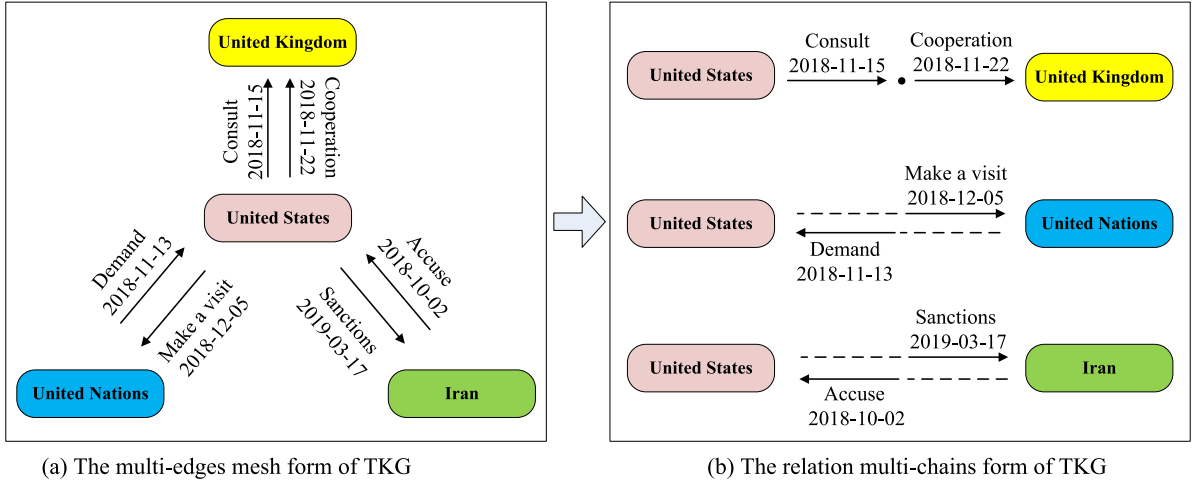


Fig. 1. An example of temporal knowledge graph, involving United Nations, United States, United Kingdom, and Iran. (a) is the multi-edges mesh form and (b) is the relational multi-chains form.

pair are clustered into a specific relational chain (e.g. (United States $\rightarrow$ “Consult/2018-11-15, Cooperation/2018-11-22” $\rightarrow$ United Kingdom)). These chains can better show the changing relation between two entities over time.

In this paper, according to relational multi-chains form, we propose a Relation-oriented Attention Network (RoAN), which focuses on modeling the temporal features of relations in TKGC. RoAN apply the attention mechanism to relations directly, which is different from existing work focusing on entities. Through RoAN, we obtain the temporal embedding of relations which can be potentially combined to most existing TKG completion models. The experimental results on three standard TKGC benchmarks ICEWS14, ICEWS05-15, and GDELT datasets show that our model can be coupled with previous TKGC methods and can increase their performance accordingly. Besides, the analysis of relation-oriented attention explains the reason of improvement. Our contributions are as follows:

- We put forward a form of relational multi-chains for temporal knowledge graphs and propose a relational-oriented attention mechanism for obtaining the temporal embedding of relations.
- We embed the temporal features of relations by applying the attention mechanism to relations directly. Besides, we potentially combined with most existing TKG completion models for TKGC task.
- We evaluate our approach on three TKGC benchmarks. The results demonstrate our approach enhance the performance of TKGC task on most metrics.

The rest of this paper is organized as follows. We make a review of related work in Section 2. Then we give the notations and details of our approach in Section 3. Experimental settings and results are reported in Section 4. In Section 5, we make a conclusion and look forward to future directions.

2. Related work

2.1. Static KG completion models

Mainstream KG completion models can be classified into two types: translational distance model and semantic matching model (Wang et al., 2017) (which is also called the bilinear model (Ji et al., 2021)).

Generally, the former measures the plausibility of a fact by the distance between two entities after the translation, such as TransE (Bordes et al., 2013), TransR (Lin et al., 2015), TransD (Ji et al., 2015) and RotatE (Sun et al., 2019). In addition, some joint learning models

exploit the additional information in knowledge graphs, such as TKRL (Xie et al., 2016a) and DKRL (Xie et al., 2016b). The latter applies the similarity-based scoring functions. They measure the likelihood of events by matching the latent semantics of entities and relations, which are embedded into the vector space representations (e.g., RESCAL (Nickel et al., 2011), DistMult (Yang et al., 2015), and HolE (Nickel et al., 2016)). Besides, ComplEx (Trouillon et al., 2016) introduces complex-valued embedding so as to better model asymmetric relations. In ComplEx, the embeddings of entities and relations no longer lie in a real space but a complex space.

In recent years, some factorization-based models (e.g., Simple (Kazemi and Poole, 2018) and TuckER (Balažević et al., 2019a)) or adopts convolution neural networks models (e.g., ConvE (Dettmers et al., 2018) and HypER (Balažević et al., 2019b)) have also been proposed. (Wang et al., 2017; Ji et al., 2021) make comprehensive reviews of these existing models, respectively.

2.2. Temporal knowledge graph completion models

In TKG reasoning, there exist two tasks, i.e., TKGC and TKG forecasting. There have been some attempts on incorporating temporal information in modeling temporal knowledge graphs, broadly categorized into two types: interpolation models and extrapolation models.

For interpolation, some work attempts to extend static KG completion models to the temporal domain. Typically, such work adopts translational distance models and design time-sensitive scoring functions. TTransE (Leblay and Chekol, 2018) adds an embedding of timestamp into scoring function. HyTE (Dasgupta et al., 2020) employs the timestamp hyperplane to project embeddings and uses the scoring function of TransE (Bordes et al., 2013) on the projected space. García-Durán et al. (2018) extend TransE (Bordes et al., 2013) and DistMult (Yang et al., 2015) by combining the textual information of timestamp through a character LSTM. Goel et al. (2020) equip static KG models with a diachronic entity embedding function which provides the characteristics of entities at any time point. Besides, some tensor-based models are also powerful. Ma et al. (2019) argue that the embeddings of temporal knowledge graph might be modeled for cognitive episodic memory (the facts we remember and can recollect), and propose ConT based on the tensor calculation. Xu et al. (2020) select to incorporate time information into entity or relation embeddings by using Additive Time Series decomposition. Wu et al. (2020) introduces frequency-based gating and data interpolation technology to deal with the time variability and sparsity problems, and the entities are represented by multi-hop structure information and time facts from nearby time steps. Lacroix et al. (2019) expands ComplEx by adding timestamp

embedding to modulate the multi-linear dot product, and extends the tensor method to TKG. Another line of interpolation work adopts the neural network to capture entity's neighbor information, which is usually combined with temporal recurrence networks or attention mechanism networks on entities (e.g. SPA (Wang et al., 2022) and ExKGR (Yan et al., 2022)). T-GAP (Jung et al., 2021) encodes query-specific substructure of TKG by focusing on the temporal displacement between each event and the query timestamp, and performs path-based reasoning by propagating attention through the graph. BoxTE (Messner et al., 2022) uses dedicated time embeddings, allowing to flexibly represent temporal information. Therefore, it introduces boxes to represent a set of answer entities to a time-agnostic query. In order to solve the problem of excessive parameters of existing models, Ding et al. (2022) propose a simple but powerful graph encoder for TKGC, called TARGCN. TARGCN is parameter-efficient, and it extensively explores every entity's temporal context for learning contextualized representations.

For extrapolation tasks, RE-Net (Jin et al., 2020) proposes a recurrent event encoder to encode historical knowledge and employs a neighborhood aggregator to model the concurrent knowledge. CluStER (Li et al., 2021a) and TITer (Sun et al., 2021) both adopt reinforcement learning to discover evolutionary patterns in query-related paths of a fixed length. Unlike the query-specific models, entire graph-based models encode the latest historical KG sequence of a fixed-length. RE-GCN (Li et al., 2021b) captures the evolutionary patterns into the representations of all the entities by modeling KG sequence of a fixed-length at latest a few timestamps. While the temporal facts are typically evolving, Zhu et al. (2021) found that many facts often show a repeated pattern along the timeline, and propose a new representation learning model, namely CyGNet, based on a novel time-aware copy-generation mechanism. CyGNet is not only able to predict future facts from the whole entity vocabulary, but also capable of identifying facts with repetition. Han et al. (2021) proposes a novel continuum model by extending the idea of neural ordinary differential equations (ODEs) to multi-relational graph convolutional networks, called TANGO. TANGO preserves the continuous nature of dynamic multi-relational graph data and encodes both temporal and structural information into continuous-time dynamic embeddings. A comprehensive analysis of these existing models for TKGC can be found in the reference Kazemi et al. (2020).

2.3. Attention mechanism of graphs

Attention mechanism is a powerful technology to distinguish different parts of input. It enhances the performance of models by assigning varying weights. Especially in recent years, self-attention mechanism (Vaswani et al., 2017) has become more and more popular. The first attempt to apply the attention mechanism to graphs is the Graph attention network (GAT) (Veličković et al., 2017), an improved method of Graph convolutional network (GCN) (Kipf and Welling, 2017). GAT learns to assign varying weights to each entity's neighbors through the attention mechanism, rather than treating all neighbors with equal importance, as is done in GCNs. In addition, the Relational Graph Attention Network (RGAT) (Busbridge et al., 2019) extend the construction of GAT layer in GAT (Veličković et al., 2017) to the relational setting, and it employs the ideas from the Relational convolutional network (RGCN) (Schlichtkrull et al., 2018) (another extension of GCN (Kipf and Welling, 2017) to the domain of relational knowledge graphs). Besides, Gated Attention Network (GAAN) (Zhang et al., 2018) collects the information from different attention heads with the self-attention mechanism to replace the average operation in GAT. Nathani et al. (2019) proposes an attention-based embedding to capture entity and relation features in any given entity's neighbors. Relational Graph neural network with Hierarchical Attention (RGHAT) (Zhang et al., 2020) is equipped with two-level attention mechanisms, relation-level attention and entity-level attention, to make better use of the local neighborhood information around the entity.

Table 1

The key notations.

Symbol	Meaning
G	The whole TKG
E, R, T	Entities, Relations, Timestamps
C_e	The relation chain of entity e
$\mathbb{R}$	The space of embeddings
Neg	Candidate set by negative sampling
k	The dimension of vectors
v_d, v_m, v_y	Vectors of day, month, year
v_e, v_r, v_t	Vectors of entity, relation, timestamp
W	The mapping matrix
H	The heads of multi-attention

3. Our method

In this section, we introduce the key notations in this paper and provide an overview of our model firstly. Then we describe the content of relation-oriented attention mechanism, and give the details of structural layers. Finally, we discuss the training of our model.

3.1. Notations and overview of our model

Notations: In this paper, we apply $G = (E, R, T)$ to denote the whole temporal knowledge graph. Here, E and R stand for the union sets of entities and relations. T denotes the set of all timestamps. Each event is denoted by the quadruple (s, r, o, t) to represent s takes the action r on o at time t , where $s \in E, o \in E, r \in R$, and $t \in T$. TKGC aims to complete incomplete events in temporal knowledge graphs. In other word, given an incomplete event $(?, r, o, t)$ or $(s, r, ?, t)$, we should predict the missing entity '?' according to existing events. Table 1 summarizes the key notations used throughout this paper for easy reference.

Overview of our model: The entire architecture of our model is shown in Fig. 2. It contains an encoder for getting temporal embeddings (the left block in Fig. 2) and a decoder for scoring events (the right block in Fig. 2). In the encoder, we map each entity $e \in E$, relation $r \in R$ and timestamp $t \in T$ to the low-dimensional embeddings v_e, v_r and v_t , and process events with the negative sampling (generate a candidate answer set which contains other entities selected randomly from E). Then we construct the temporal embeddings for relations and entities. The temporal embeddings of relations are obtained by our approach RoAN and the temporal embeddings of entities can be calculated by current temporal completion models. In the decoder, we adopt the raw scoring function of base models to estimate the likelihood of events. Besides, cross-entropy is applied to calculate the training loss.

3.2. Relation-oriented attention mechanism

Though attention mechanism has been widely used in KG completion models, these models mainly focus on entity features and they cannot handle the temporal information of relations where relations between two entities are changing over time. In order to solve this problem, we apply the attention mechanism to relations rather than entities for embedding the temporal features of relations.

As discussed in the introduction, the form of relational multi-chains is the base of our relation-oriented attention mechanism. For each entity, we cluster its linked relations as a chain, which is represented as C_e . It is noting that one entity may play different roles (the subject entity or the object entity) in the events and that a relation contains different effects to its end entities. Therefore, we build two relation chains for each entity according to its role in the event (subject or object). As shown in Fig. 3, there are two different relation chains for the entity "United States", one is the chain for subject role and another is for object role. Given an incomplete event (United States, provide aid, ?, 2019-03-06), we employ the subject chain according to the role of "United States". Then we calculate the attention of "provide aid" to

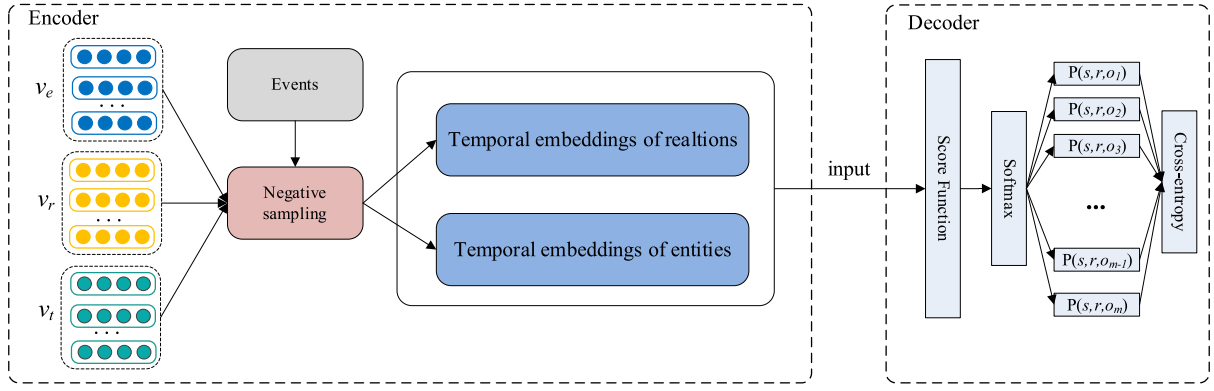


Fig. 2. The overall architecture of RoAN. v_e represent entity embedding vectors, v_r represent relation embedding vectors, and v_t represent timestamp embedding vectors.

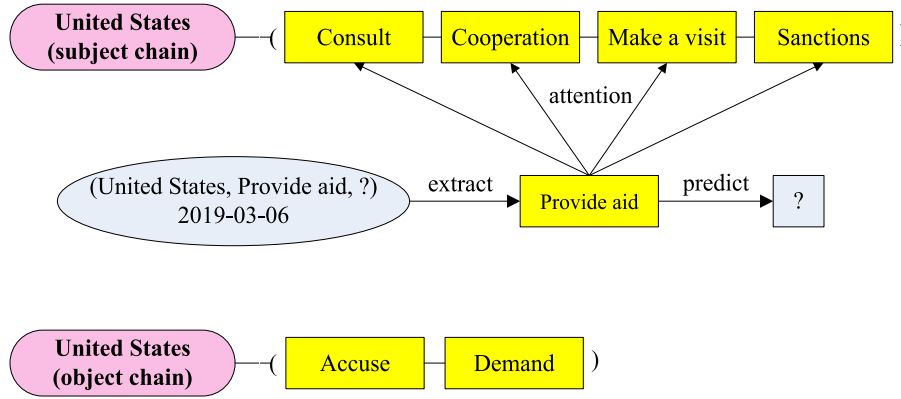


Fig. 3. The illustration of relation-oriented attention mechanism. Rounded rectangles represent entities with different roles and rectangles represent relations.

other relations in the chain. Through our relation-oriented attention mechanism, model can focus on relevant relations of “provide aid” based on the known entity “United States” for prediction.

Meanwhile, in order to handle the temporal information of events, we construct dynamic matrix spaces with the embeddings of entities and timestamps. By mapping relations to these spaces, we can obtain the different representations of relations under specific entity and timestamp. The entire architecture of Relation-oriented Attention mechanism is shown in Fig. 4. It includes three layers Temporal Mapping layer, Attention Layer, and Feed-Forward Layer. At first, in Temporal Mapping Layer, we construct a matrix space W_{et} according to the vectors of entity and timestamp, and map relations to handle the temporal information of events. Then, we calculate the attention of relations in Attention Layer, which is different from existing work focusing on entities. The Feed-Forward Layer encodes the vector after Attention Layer for following operation.

3.3. Structural layers

There are three layers in our relation-oriented attention mechanism, including Temporal Mapping Layer, Attention Layer, and Feed-Forward Layer. Given an incomplete event (s, r, o, t) , we obtain two sub prediction tasks $(s, r, ?, t)$ and $(?, r, o, t)$. In the following content, we select $(s, r, ?, t)$ as the target task to illustrate the details of our approach. Similar procedure is applied to $(?, r, o, t)$, except the initial entity is object o .

Temporal Mapping Layer In order to handle the temporal information of event, we construct matrix spaces according to the vectors of entities and timestamps, and then map relations for particular representations. At first, in order to reduce the parameters of timestamps, we split each timestamp “Years-Months-Days” to (“Years”, “Months”,

“Days”), and embed them into low-dimensional vectors v_y , v_m and v_d . Then, the simplest addition is applied to combine v_y , v_m and $v_d \in \mathbb{R}^k$ as the timestamp vector (Eq. (1)).

$$v_t = v_y + v_m + v_d \quad (1)$$

Then, we calculate the transform matrix W_{et} by v_s and v_t to construct the mapping space as shown in Eq. (2).

$$W_{et} = v_s v_t^T \quad (2)$$

where $v_s, v_t \in \mathbb{R}^k$ are the vectors of entity s and timestamp t , respectively. T means the transpose operation.

Considering that we only know the information of s , so we cluster relevant relations of s as the chain represented as C_e . Then we map each relation vector $v_{r_i} \in \{r \cup C_e\}$ to the constructed space $W_{et} \in \mathbb{R}^{k \times k}$ (Eq. (3)).

$$v'_{r_i} = W_{et} v_{r_i} \quad r_i \in r \cup C_e \quad (3)$$

where $v'_{r_i} \in \mathbb{R}^k$ is the vector of relation r_i . $W_{et} \in \mathbb{R}^{k \times k}$ is the linear transformation matrix based on the vectors of entity and timestamp.

Attention Layer: In Attention layer, we employ “Scaled Dot-Product Attention” and “Multi-head Attention” techniques, which are proposed by Vaswani et al. (2017) to calculate the attention between relations. In practice, the query is represented by v'_r , the vector of relation r . The keys and values of attention mechanism are packed together into matrixes K_{C_e} and V_{C_e} according to the chain C_e . The output of attention layer is represented as v''_r (Eq. (4)).

$$v''_r = \text{attn}(v'_r, K_{C_e}, V_{C_e}) \quad (4)$$

where $\text{attn}()$ is the calculation process of attention mechanism as follows.

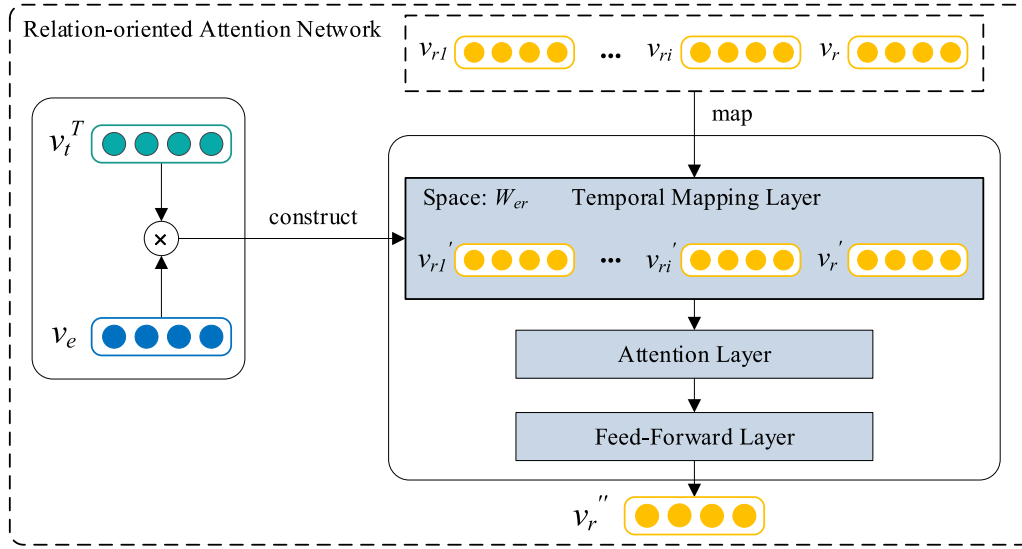


Fig. 4. The overall architecture of relation-oriented attention mechanism. v_e represents entity embedding vector, v_r represents relation embedding vector, and v_t represents timestamp embedding vector. W_{er} is a matrix space constructed by the vectors of entity and timestamp.

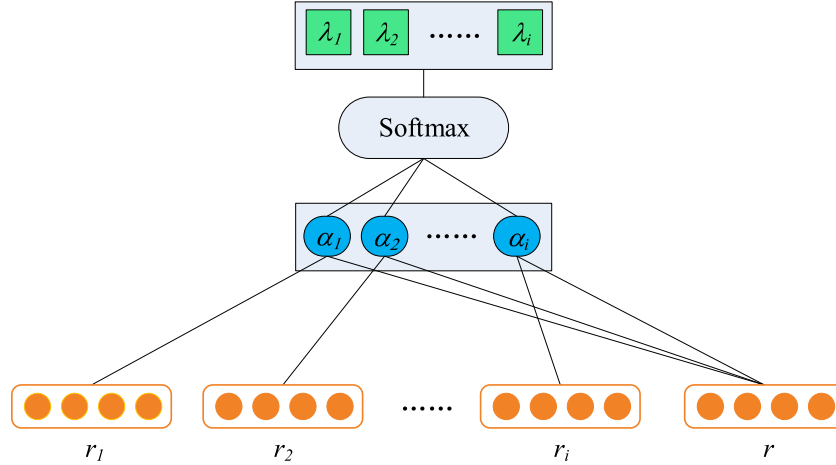


Fig. 5. The computation of Attention.

Referring to the “Scaled Dot-Product Attention”, we employ a multiplication between v'_r and v'_{r_i} for each $r_i \in C_e$ to calculate the attention of relations, and divide the values by $\sqrt{k}$ for scaling, where n' is the dimension of temporal embedding (Eq. (5)). Attention value represents the importance of each relation $r_i \in C_e$ for the source relation r , and it is denoted by α_i . The relative attention λ_i is computed by the Softmax function over all attention values as shown in Eq. (6). Fig. 5 illustrates the computation of λ_i for each relation in the chain.

$$\alpha_i = \frac{(v'_r{}^T v'_{r_i})}{\sqrt{k}} \quad (5)$$

$$\lambda_i = \text{Softmax}(\alpha_i) = \frac{\exp(\alpha_i)}{\sum_{r_k \in C_e} \exp(\alpha_k)} \quad (6)$$

where v'_r and $v'_{r_i} \in \mathbb{R}^k$ denote the mapped vectors of r and r_i , respectively.

The new vector of r is the sum of all relation vectors in the chain weighted by their attention values as shown in Eq. (7).

$$v''_r = \sum_{r_i \in C_e} \lambda_i v'_{r_i} \quad (7)$$

Besides, as suggested by Busbridge et al. (2019), the “Multi-head attention” is used to encapsulate more information about the relations in the learning process. H independent attention heads are used to calculate the embeddings, which are then concatenated as the following representation (Eq. (8)).

$$v''_r = \prod_{h=1}^H \sum_{r_i \in C_e} \lambda_i^h v'_{r_i} \quad (8)$$

where Π represents the concatenation operation.

Feed-Forward Layer: In addition to Attention Layer, there is a full connected feed-forward layer as Vaswani et al. (2017) designed. It consists of two linear transformations with an activation function gelu (Hendrycks and Gimpel, 2016) in between (Eq. (9)).

$$\bar{v}_r = \sigma(v''_r W_f + b_f) W_{ff} + b_{ff} \quad (9)$$

where $W_f \in \mathbb{R}^{k \times 2k}$ and $W_{ff} \in \mathbb{R}^{2k \times k}$ are the linear transformation matrixes.

Besides, Res-Net (He et al., 2016) and Layer Normalization (Ba et al., 2016) are employed to get the final encoded vector as shown in Eq. (10). The Dropout technique (Srivastava et al., 2014) is also

applied to avoid the overfitting problem.

$$\bar{v}_r = LN(\bar{v}_r + v_r'') \quad (10)$$

where $LN()$ is the Layer Normalization function.

In the end, we adopt the weighted sum of v_r and $\bar{v}_r$ as the final embeddings of relations as shown in Eq. (11).

$$v_{rt} = (1 - \beta)v_r + \beta\bar{v}_r \quad (11)$$

where $\beta \in [0, 1]$ is the weight of temporal embeddings.

3.4. Decoder

In the training, we extend DE-Simple model (Goel et al., 2020), the latest temporal translational models, by combining with our RoAN approach. In the following, we call the extended model as RoAN-DES. For entity embeddings, we employ the implement of DE-Simple (Goel et al., 2020) as shown in Eq. (12). We refer the reader to the reference Goel et al. (2020) for more details.

$$v_{et} = DE(v_e, t) = v_{e[tdim]} \prod v_{e[tdim]} \sigma(w_t t + b_t) \quad (12)$$

where $DE()$ is the embedded function of entities, $tdim$ is the dimension of temporal part.

Now, all embeddings of entities and relations contain the temporal information. The scoring function following the original form of Simple (Kazemi and Poole, 2018) is applied to calculate the likelihood of events in the decoder (Eq. (13)).

$$Sco(v_s, v_o', v_r) = \text{sum} \left((v_{st} * v_{rt} * v_{ot}) + (v_{st} * v_{rt} * v_{ot})^{-1} \right) \quad (13)$$

where $\text{sum}()$ is the addition of each elements in the vector, and -1 is the inverse of event.

For the task $(s, r, ?, t)$, we generate a candidate answer set Neg which is composed of training triplets with either the head or tail replaced by a random entity (but not both at the same time). Besides, the *cross-entropy* is applied as loss function for training (Eq. (14)). For the second task $(?, r, o, t)$, we adopt the same calculation. Our approach aims to minimize the whole loss as shown in Eq. (15):

$$L(s, r, t) = - \sum_{(s, o, t) \in G_{train}} \frac{\exp(Sco(v_s, v_o', v_r)_o)}{\sum_{i \in Neg} \exp(Sco(v_s, v_o', v_r)_i)} \quad (14)$$

$$L = L(s, r, t) + L(o, r, t) \quad (15)$$

where G_{train} is the train-set, and Neg is the candidate set by negative sampling.

The complete procedure of RoAN-DES is shown in Algorithm 1.

4. Experiments

4.1. Datasets and baselines

Datasets: We apply Global Database of Events, Language and Tone (GDELT) (Leblay and Chekol, 2018) and Integrated Crisis Early Warning System (ICEWS) (Boschee et al., 2015) datasets as our benchmarks. For ICEWS, we use the two subsets generated by García-Durán et al. (2018): ICEWS14 includes the events in 2014 and ICEWS05-15 contains all events from 2005 to 2015. For GDELT, we use the subset extracted by Trivedi et al. (2017) corresponding to the events from April 1, 2015 to March 31, 2016. We apply the same partitioning of train-set, valid-set and test-set as specified by the reference Goel et al. (2020). The dataset statistics are summarized in Table 2.

Baselines: We compare our model against several static KGC models and existing state-of-the-art translational models for TKGC (interpolation task). For static KGC models, we include TransE (Bordes et al., 2013), DistMult (Yang et al., 2015) and Simple (Kazemi and Poole, 2018). Note that we neglect the time information in the input for all static baseline models. We also compare with state-of-the-art translational models designed for TKGC including TTransE (Leblay and Chekol, 2018), HyTe (Dasgupta et al., 2018), TA models (García-Durán et al., 2018) (TA-TransE and TA-DistMult), and DE models (Goel et al., 2020) (DE-TransE, DE-DistMult and DE-Simple).

4.2. Evaluation metrics

For each event (s, r, o, t) in the test set, we evaluate two sub prediction tasks $(s, r, ?, t)$ and $(?, r, o, t)$. We report the *Hits@1*, *Hits@3*, *Hits@10* scores and *MRR* (mean reciprocal rank) as metrics. For each task, we return top- i entities as the candidate set. When the target entity exists in them yielding as *Hits@i*, where $i = 1, 3, 10$. *MRR* is the mean reciprocal ranking of all target entities. Higher *Hits@i* and *MRR* is expected. Similar to baselines, evaluation is performed under the filtered setting defined by Bordes et al. (2013) (those corrupted quadruples existed in TKG should be filter out before the ranking, where $\{(s, r, o', t) \mid (s, r, o, t) \in G\}$ for task $(s, r, ?, t)$ or $\{(s', r, o, t) \mid (s', r, o, t) \in G\}$ for task $(?, r, o, t)$).

4.3. Implementation and hyperparameters

We implemented our model in PyTorch. In experiments, we combined our RoAN approach with the DE models (Goel et al., 2020) and follow their raw settings. We set the number of multi-head attention $H = 4$. We selected the embedding dimension $k = 100$, the temporal dimension $tdim$ among $\{36, 64\}$, the batch size B among $\{512, 1024\}$, the learning rate λ among $\{0.001, 0.0001\}$, and the rate γ of Dropout layer among $\{0.0, 0.2, 0.4\}$. The weight of relation embedding is set as $\beta = 0.5$. Besides, we set the negative ratio = 500 for ICEWS datasets and negative ratio = 5 for GDELT dataset as DE models (Goel et al., 2020) did. Adam (Kingma and Ba, 2014) is selected as the optimizer (weight decay = 0.0). The max epoch is restricted to 500 and the optimal hyperparameters is selected according to *MRR* on the validation set. The best configurations across three benchmarks are as follows: $\{k = 100, tdim = 64, B = 512, \lambda = 0.001, \gamma = 0.4\}$ on ICEWS14 dataset, $\{k = 100, tdim = 64, B = 512, \lambda = 0.001, \gamma = 0.4\}$ on ICEWS05-15 dataset, and $\{k = 100, tdim = 32, B = 512, \lambda = 0.001, \gamma = 0.4\}$ on GDELT dataset. We ran our experiments on a node with two GeForce GTX 2080 Ti.

4.4. Results and analysis

(1) Comparative Study

We experiment three variants RoAN-DET, RoAN-DED and RoAN-DES by combining our RoAN approach with latest temporal translational models DE-TransE, DE-DistMult and DE-Simple. Results on each dataset are obtained by averaged metrics (5 runs) under the optimal configurations. The core experimental results compared with the baselines are summarized in Table 3. We find RoAN can be coupled with previous TKGC methods and can increase their performance accordingly.

Our results show that RoAN-DES obtains the best experimental results due to its higher expressivity. Compared with the raw DE-Simple (Goel et al., 2020), we get following improvements: Across the dataset ICEWS14, RoAN-DES have almost 5.8% improvement than original DE-Simple model on the *MRR*, *Hits@1*, *Hits@3* and *Hits@10* metrics; Across the ICEWS05-15 datasets, our model obtains achieve 8.5% average relative improvement on all metrics; Across the GDELT dataset, we obtain almost 6.0% improvement on most metrics, especially outperforms 9% on *Hits@10*. These results highlight the ability of RoAN to improve the performance of TKGC task by applying the relation-oriented attention mechanism, and demonstrate the power of relations in TKGC task.

Besides, the models combined with our RoAN method (RoAN-DET, RoAN-DED and RoAN-DES) all outperform other corresponding baselines across three datasets. Especially, our TransE-based method RoAN-DET beats other DistMult-based baselines on most metrics and gets better results on *Hits@10* than DE-Simple model. It demonstrates our method RoAN can enhance the performance of most translational models and have a strong applicability for TKGC.

ALGORITHM 1 The Procedure of RoAN-DES method

Input: Train set G_{train} , Relational multi-chains R , Window length w , Epoch q , Loss $L = 0.0$

Output: The minimum Loss L_{min} on the train set

01 $v_s, v_r, v_o, v_t \leftarrow \text{xavier-uniform}()$, $L_{min} = MAXINT$ // Initialize embedding vectors and minimum Loss

02 **for** $epoch \leftarrow 1$ **to** q **do**

03 $L \leftarrow 0.0$ // Initialize the Loss

04 **for** $(s, r, o, t) \in G_{train}$ **do**

05 split (s, r, o, t) into two tasks $(s, r, ?, t)$ and $(?, r, o, t)$

06 **for** each task in $\{(s, r, ?, t), (?, r, o, t)\}$

07 map relations to the matrix space of entity and timestamp by **Equation (1)(2)(3)**

08 calculate the attention between relations by **Equation (5)(6)(7)(8)**

09 encode the final relation vector through Feed-forward Layer by **Equation (9)(10)(11)**

10 calculate $Sco(v_s, v'_o, v_r)$ and $Sco(v_o, v'_s, v_r)$ by **Equation (13)**

11 calculate $L_{(s, r, t)}$ and $L_{(o, r, t)}$ by **Equation (14)**

12 $L = L(s, r, t) + L(o, r, t)$

13 $L_{min} = \min(L_{min}, L)$

14 Update parameters v_s, v_r, v_o with Adam (Kingma and Ba, 2014) optimizer

15 **return** L_{min}

Table 2
Statistics of datasets.

Dataset	#Entities	#Relations	#years $\times$ months	#Train	# Valid	#Test
ICEWS14	7128	230	$1 \times 12 \times 31$	72,826	8,941	8,963
ICEWS05-15	10488	251	$10 \times 12 \times 31$	386,962	46,275	46,092
GDELT	500	20	$2 \times 12 \times 31$	2,735,685	341,961	341,961

Table 3
Temporal KG completion evaluation results on ICEWS, ICEWS05-15 and GDELT. The Hit@1, Hit@3, and Hit@10 metrics are multiplied by 100.

Model	ICEWS14				ICEWS05-15				GDELT			
	MRR	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10	MRR	Hit@1	Hit@3	Hit@10
TransE ^a	0.280	9.4	–	63.7	0.294	9.0	–	66.3	0.113	0.0	15.8	31.2
DistMult ^a	0.439	32.3	–	67.2	0.456	33.7	–	69.7	0.196	11.7	20.8	34.8
SimpleE ^b	0.458	34.1	51.6	68.7	0.478	35.9	53.9	70.8	0.206	12.4	22.0	36.6
TTransE ^b	0.255	7.4	–	60.1	0.271	8.4	–	61.6	0.115	0.0	16.0	31.8
HyTE ^b	0.297	10.8	41.6	65.5	0.316	11.6	44.5	68.1	0.118	0.0	16.5	32.6
TA-TransE ^a	0.275	9.5	–	62.5	0.299	9.6	–	66.8	–	–	–	–
TA-DistMult ^b	0.477	36.3	–	68.6	0.474	34.6	–	72.8	0.206	12.4	21.9	36.5
DE-TransE ^b	0.326	12.4	46.7	68.6	0.314	10.8	45.3	68.5	0.126	0.0	18.1	35.0
DE-DistMult ^b	0.501	39.2	56.9	70.8	0.484	36.6	54.6	71.8	0.213	13.0	22.8	37.6
DE-SimpleE ^b	0.526	41.8	59.2	72.5	0.513	39.2	57.8	74.8	0.230	14.1	24.8	40.3
RoAN-DET	0.461	29.2	57.7	76.1	0.477	30.4	59.3	78.4	0.205	0.081	25.1	43.5
RoAN-DED	0.569	45.7	64.4	77.4	0.568	45.0	63.8	79.1	0.268	16.9	28.9	46.4
RoAN-DES	0.588	47.6	66.1	78.8	0.599	47.9	67.9	82.3	0.290	18.7	31.5	49.6

Best Results are in Bold. Dashes: Results are Unobtainable.

^aResults are taken from the reference García-Durán et al. (2018).

^bResults are taken from the reference Goel et al. (2020).

Table 4
Experimental results for different variations of our model.

Dataset	Variants	MRR %	Hits@1%	Hits@3%	Hits@10%
ICEWS14	Scaled Dot-product	0.588	47.6	66.1	78.8
	Dot-product function	0.536	47.0	59.3	78.5
	General function	0.538	47.1	59.6	78.6
ICEWS05-15	Scaled Dot-product	0.599	47.9	67.9	82.3
	Dot-product function	0.548	47.4	61.2	81.9
	General function	0.549	47.4	61.3	82.0
ICEWS14	(“Years” + “Months” + “Days”)	0.588	47.6	66.1	78.8
	(“Months” + “Days”)	0.587	46.4	65.5	78.3
	(“Years” + “Months”)	0.586	46.3	65.3	77.9
ICEWS05-15	(“Years” + “Months” + “Days”)	0.599	47.9	67.9	82.3
	(“Months” + “Days”)	0.596	46.9	67.4	81.9
	(“Years” + “Months”)	0.595	46.8	67.2	81.4

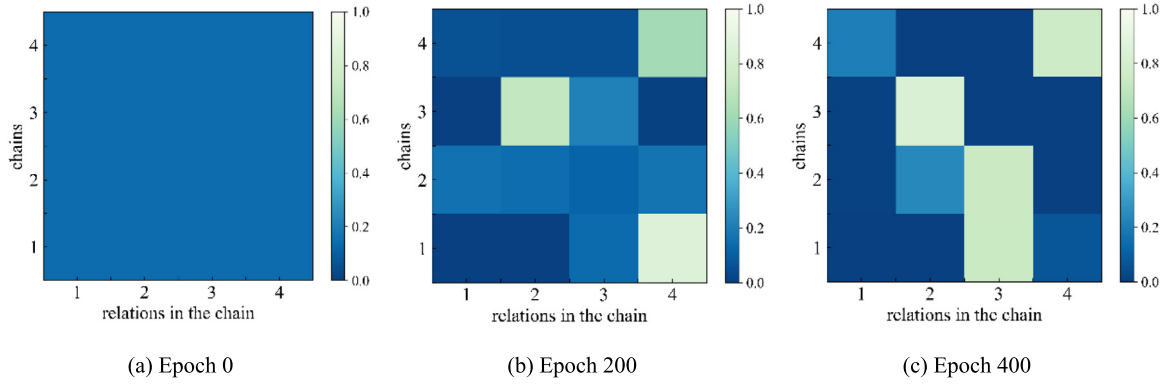


Fig. 6. Learning the attention of our approach among epochs on ICEWS14 dataset. Y-axis is the related chain of one given relation, and X-axis represents the index of relation in a chain. The lighter block contains higher attention.

(2) Analysis of Relation-oriented Attention mechanism

Because the relation codes of three benchmarks all follow the CAMEO taxonomy (Schrodt, 2012) which contains over 300 different types coded relations, and our attention mechanism mainly focuses on relations, the distribution of attention for all benchmarks will be the same. In this subsection, we select ICEWS14 dataset to analyze our relation-oriented attention mechanism. Firstly, we draw the attention of a given relation to four related chains (each chain contains four relations) with increasing epochs. The result is shown in Fig. 6. In the initial Epoch (0) of the training process, the attention is distributed discretely. As the epoch increases, our approach gathers more information from relation chains. It assigns more attention to the relevant relations of query relation in each chain (the lighter blocks in Fig. 6) and takes less information from others. Once the training process finishes, our model learns to cluster relevant relations information from the chain of each entity at specific timestamp.

Besides, we show the distribution of relations for four entity pairs. The results are presented in Fig. 7(a). The lightest block in each chain (row) represents the most frequent relation between two entities. Then for each chain, we draw the attention from the lightest block to others as shown in Fig. 7(b). We can see that the lightest blocks in Fig. 7(b) are also lighter blocks in Fig. 7(a) mostly (the circled part in red). It means RoAN architecture can teach a relation to focus on relevant relations for each entity pair.

(3) Model Variants and Ablation Study

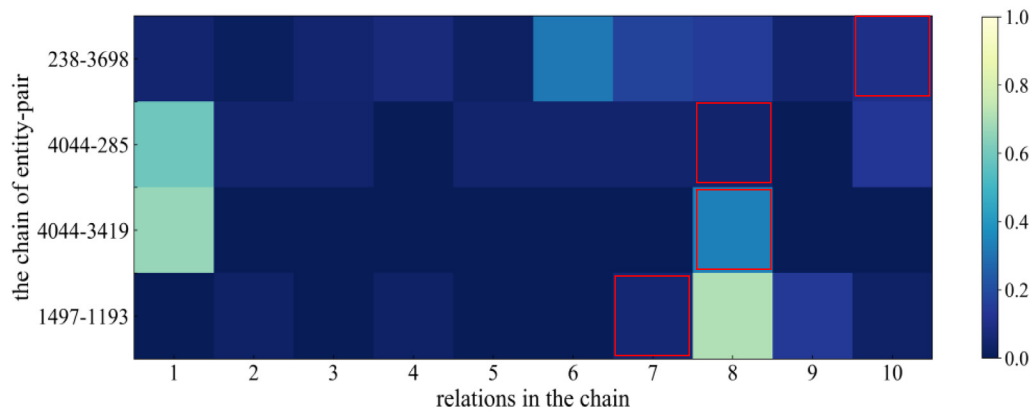
Attention Function So far, we applied the “Scaled Dot-product” function in Attention Layer. There are still some attention functions available for us, including “Dot-product” function and “General” function (Luong et al., 2015). The performances across ICEWS14 and ICEWS05-15 datasets are presented in Table 4 (corresponding lines of ICEWS14 and ICEWS05-15). It can be viewed that other attention functions also perform well. The “General” activation slightly worse

than “Scaled Dot-product” function. “Dot-product” has an impact on the performance of our model, because it has a larger range of attention values, which leads great fluctuation during the training process and affects the model convergence.

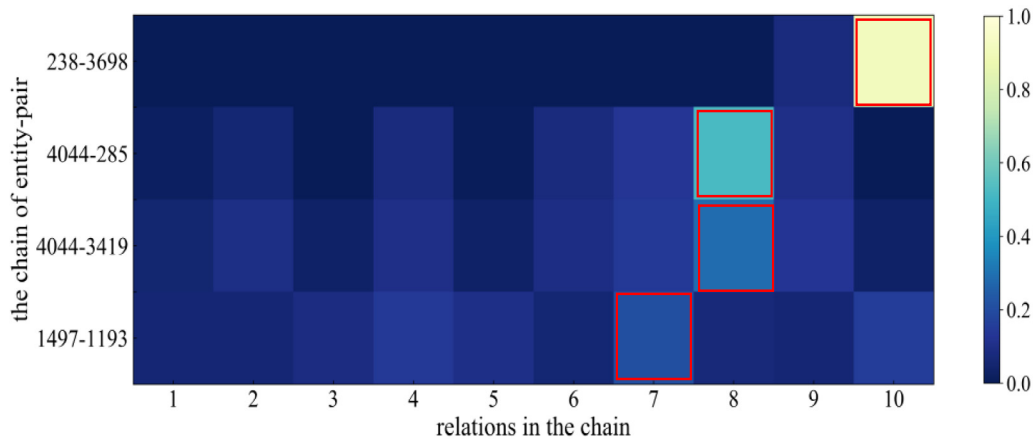
Time Granularities: In the temporal mapping layer part, we split each timestamp to (“Years”, “Months”, “Days”), and apply (“Years” + “Months” + “Days”) to construct the embedding of timestamp. In order to verify the effect of time granularities, we test model variants with other configurations including (“Months” + “Days”) and (“Years” + “Months”). Considering that ICEWS14 and ICEWS05-15 contains more temporal information than other benchmarks, we experiment these variants on it. From the obtained results (corresponding lines of ICEWS14 and ICEWS05-15), one can see that the rich temporal information will enhance the performance of RoAN. We believe one reason why the fine temporal granularities give better performance is because it ensures that there is enough temporal information of relations to be taken into account in the coding process.

5. Conclusion

In this paper, we proposed a novel method RoAN (Relation-oriented Attention Network) for temporal knowledge graph completion task. We adopt a form of relational multi-chains to reconstruct TKGs. What is more, we applied the attention mechanism to relations rather than entities, which is different from existing models. It is worth noting that RoAN is model-agnostic and can be potentially combined with most existing TKG completion models. The experimental results show that RoAN-based models achieved improvement on many evaluation indicators across three benchmarks. Besides, through the analysis, we explained how the relation-oriented attention works and why RoAN performs better.



(a) The distribution of relations



(b) The distribution of attentions

Fig. 7. The distribution of relations and attention, respectively. (a) is the relations distribution of four entity pairs on ICEWS14 dataset and (b) is the attention distribution from the most frequent relation to others for a chain (row). Y-axis is the chain of each entity pair and X-axis represents the index of relation in the chain.

Future work involves enhancing the performance of RoAN by combining with previous attention-based methods which focus on entity features and extending our approach to TKGs extrapolation task.

CRedit authorship contribution statement

Luyi Bai: Conceptualization, Methodology, Formal analysis, Funding acquisition, Writing – original draft, Writing – review & editing. **Xiangnan Ma:** Investigation, Formal analysis, Writing – original draft. **Xiangxi Meng:** Validation, Writing – review & editing. **Xin Ren:** Formal analysis, Writing – review & editing. **Yujing Ke:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The work was supported by the National Natural Science Foundation of China (61402087), the Natural Science Foundation of Hebei Province, China (F2022501015), the Key Project of Scientific Research Funds in Colleges and Universities of Hebei Education Department (ZD2020402), the Fundamental Research Funds for the Central Universities, China (N2023019), and in part by the Program for 333 Talents in Hebei Province (A202001066).

References

- Ba, J.L., Kiros, J.R., Hinton, G.E., 2016. Layer normalization. arXiv preprint [arXiv:1607.06450](https://arxiv.org/abs/1607.06450).
- Balažević, I., Allen, C., Hospedales, T., 2019a. TuckER: Tensor factorization for knowledge graph completion. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. EMNLP-IJCNLP, pp. 5185–5194. [http://dx.doi.org/10.18653/v1/D19-1522](https://doi.org/10.18653/v1/D19-1522).
- Balažević, I., Allen, C., Hospedales, T.M., 2019b. Hypernetwork knowledge graph embeddings. In: Artificial Neural Networks and Machine Learning ICANN 2019: Workshop and Special Sessions: 28th International Conference on Artificial Neural Networks, Munich, Germany, September 17–19, 2019, Proceedings, Vol. 28. pp. 553–565. [http://dx.doi.org/10.1007/978-3-030-30493-5_52](https://doi.org/10.1007/978-3-030-30493-5_52).
- Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., Yakhnenko, O., 2013. Translating embeddings for modeling multi-relational data. *Adv. Neural Inf. Process. Syst.* 26.
- Boschee, E., Lautenschlager, J., O'Brien, S., Shellman, S., Starz, J., Ward, M., 2015. Integrated crisis early warning system (icews) coded event data. URL: <https://dataverse.harvard.edu/dataverse/icews>.

- Busbridge, D., Sherburn, D., Cavallo, P., Hammerla, N.Y., 2019. Relational graph attention networks. arXiv preprint [arXiv:1904.05811](https://arxiv.org/abs/1904.05811).
- Cao, Y., Wang, X., He, X., Hu, Z., Chua, T., 2019. Unifying knowledge graph learning and recommendation: Towards a better understanding of user preferences. In: The World Wide Web Conference. pp. 151–161. [http://dx.doi.org/10.1145/3308558.3313705](https://doi.org/10.1145/3308558.3313705).
- Dasgupta, S.S., Ray, S.N., Talukdar, P., 2018. Hyte: Hyperplane-based temporally aware knowledge graph embedding. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. EMNLP 2018, pp. 2001–2011. [http://dx.doi.org/10.18653/v1/D18-1225](https://doi.org/10.18653/v1/D18-1225).
- De Winter, S., Decuyper, T., Mitrović, S., Baesens, B., De Weerd, J., 2018. Combining temporal aspects of dynamic networks with Node2Vec for a more efficient dynamic link prediction. In: 2018 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining. ASONAM, pp. 1234–1241. [http://dx.doi.org/10.1109/ASONAM.2018.8508272](https://doi.org/10.1109/ASONAM.2018.8508272).
- Dettmers, T., Minervini, P., Stenetorp, P., Riedel, S., 2018. Convolutional 2d knowledge graph embeddings. In: Proceedings of the AAAI Conference on Artificial Intelligence. [http://dx.doi.org/10.1609/aaai.v32i1.11573](https://doi.org/10.1609/aaai.v32i1.11573).
- Ding, Z., Ma, Y., He, B., Han, Z., Tresp, V., 2022. A simple but powerful graph encoder for temporal knowledge graph completion. In: NeurIPS 2022 Temporal Graph Learning Workshop.
- Do, P., Phan, T.H., 2022. Developing a BERT based triple classification model using knowledge graph embedding for question answering system. Appl. Intell. 52, 636–651. [http://dx.doi.org/10.1007/s10489-021-02460-w](https://doi.org/10.1007/s10489-021-02460-w).
- García-Durán, A., Dumančić, S., Niepert, M., 2018. Learning sequence encoders for temporal knowledge graph completion. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. pp. 4816–4821. [http://dx.doi.org/10.18653/v1/D18-1516](https://doi.org/10.18653/v1/D18-1516).
- Goel, R., Kazemi, S.M., Brubaker, M., Poupart, P., 2020. Diachronic embedding for temporal knowledge graph completion. In: Proceedings of the AAAI Conference on Artificial Intelligence. pp. 3988–3995. [http://dx.doi.org/10.1609/aaai.v34i04.5815](https://doi.org/10.1609/aaai.v34i04.5815).
- Han, Z., Ding, Z., Ma, Y., Gu, Y., Tresp, V., 2021. Learning neural ordinary equations for forecasting future links on temporal knowledge graphs. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 8352–8364. [http://dx.doi.org/10.18653/v1/2021.emnlp-main.658](https://doi.org/10.18653/v1/2021.emnlp-main.658).
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 770–778. [http://dx.doi.org/10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- Hendrycks, D., Gimpel, K., 2016. Gaussian error linear units (gelus). arXiv preprint [arXiv:1606.08415](https://arxiv.org/abs/1606.08415).
- Ji, G., He, S., Xu, L., Liu, K., Zhao, J., 2015. Knowledge graph embedding via dynamic mapping matrix. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). pp. 687–696. [http://dx.doi.org/10.1007/978-3-030-01391-2_18](https://doi.org/10.1007/978-3-030-01391-2_18).
- Ji, S., Pan, S., Cambria, E., Marttinen, P., Philip, S.Y., 2021. A survey on knowledge graphs: Representation, acquisition, and applications. IEEE T. Neur. Net. Lear. 33, 494–514. [http://dx.doi.org/10.1109/TNNLS.2021.3070843](https://doi.org/10.1109/TNNLS.2021.3070843).
- Jiang, T., Liu, T., Ge, T., Sha, L., Chang, B., Li, S., Sui, Z., 2016. Towards time-aware knowledge graph completion. In: Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers. pp. 1715–1724.
- Jin, W., Qu, M., Jin, X., Ren, X., 2020. Recurrent event network: Autoregressive structure inference over temporal knowledge graphs. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. EMNLP, pp. 6669–6683. [http://dx.doi.org/10.18653/v1/2020.emnlp-main.541](https://doi.org/10.18653/v1/2020.emnlp-main.541).
- Jung, J., Jung, J., Kang, U., 2021. Learning to walk across time for interpretable temporal knowledge graph discovery. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. pp. 786–795. [http://dx.doi.org/10.1145/3447548.3467292](https://doi.org/10.1145/3447548.3467292).
- Kazemi, S.M., Goel, R., Jain, K., Kobayez, I., Sethi, A., Forsyth, P., Poupart, P., 2020. Representation learning for dynamic graphs: A survey. J. Mach. Learn. Res. 21, 2648–2720.
- Kazemi, S.M., Poole, D., 2018. Simple embedding for link prediction in knowledge graphs. Adv. Neural Inf. Process. Syst. 31.
- Kingma, D.P., Ba, J., 2014. Adam: A method for stochastic optimization. arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations.
- Lacroix, T., Obozinski, G., Usunier, N., 2019. Tensor decompositions for temporal knowledge base completion. In: International Conference on Learning Representations.
- Leblay, J., Chekol, M.W., 2018. Deriving validity time in knowledge graph. In: Companion Proceedings of the Web Conference 2018. pp. 1771–1776. [http://dx.doi.org/10.1145/3184558.3191639](https://doi.org/10.1145/3184558.3191639).
- Leetaru, K., Schrod, P.A., 2013. Gdelt: Global data on events, location, and tone, 1979–2012. In: ISA Annual Convention. pp. 1–49.
- Li, Z., Jin, X., Guan, S., Li, W., Guo, J., Wang, Y., Cheng, X., 2021a. Search from history and reason for future: Two-stage reasoning on temporal knowledge graphs. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). pp. 4732–4743. [http://dx.doi.org/10.18653/v1/2021.acl-long.365](https://doi.org/10.18653/v1/2021.acl-long.365).
- Li, Z., Jin, X., Li, W., Guan, S., Guo, J., Shen, H., Wang, Y., Cheng, X., 2021b. Temporal knowledge graph reasoning based on evolutionary representation learning. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 408–417. [http://dx.doi.org/10.1145/3404835.3462963](https://doi.org/10.1145/3404835.3462963).
- Lin, Y., Liu, Z., Sun, M., Liu, Y., Zhu, X., 2015. Learning entity and relation embeddings for knowledge graph completion. In: Proceedings of the AAAI Conference on Artificial Intelligence. [http://dx.doi.org/10.1609/aaai.v29i1.9491](https://doi.org/10.1609/aaai.v29i1.9491).
- Luong, T., Pham, H., Manning, C.D., 2015. Effective approaches to attention-based neural machine translation. In: Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. EMNLP 2015, Lisbon, Portugal, September 17–21, 2015, pp. 1412–1421. [http://dx.doi.org/10.18653/v1/D15-1166](https://doi.org/10.18653/v1/D15-1166).
- Ma, Y., Tresp, V., Daxberger, E.A., 2019. Embedding models for episodic knowledge graphs. J. Web Semant. 59, 100490. [http://dx.doi.org/10.1016/j.websem.2018.12.008](https://doi.org/10.1016/j.websem.2018.12.008).
- Messner, J., Abboud, R., Ceylan, I.I., 2022. Temporal knowledge graph completion using box embeddings. In: Proceedings of the AAAI Conference on Artificial Intelligence. pp. 7779–7787. [http://dx.doi.org/10.1609/aaai.v36i7.20746](https://doi.org/10.1609/aaai.v36i7.20746).
- Nathani, D., Chauhan, J., Sharma, C., Kaul, M., 2019. Learning attention-based embeddings for relation prediction in knowledge graphs. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. pp. 4710–4723. [http://dx.doi.org/10.18653/v1/P19-1466](https://doi.org/10.18653/v1/P19-1466).
- Nguyen, G.H., Lee, J.B., Rossi, R.A., Ahmed, N.K., Koh, E., Kim, S., 2018. Continuous-time dynamic network embeddings. In: Companion Proceedings of the Web Conference 2018. pp. 969–976. [http://dx.doi.org/10.1145/3184558.3191526](https://doi.org/10.1145/3184558.3191526).
- Nickel, M., Rosasco, L., Poggio, T., 2016. Holographic embeddings of knowledge graphs. In: Proceedings of the AAAI Conference on Artificial Intelligence. [http://dx.doi.org/10.1609/aaai.v30i1.10314](https://doi.org/10.1609/aaai.v30i1.10314).
- Nickel, M., Tresp, V., Kriegel, H., 2011. A three-way model for collective learning on multi-relational data. In: IJML. pp. 310482–3104584.
- Schlichtkrull, M., Kipf, T.N., Bloem, P., Van Den Berg, R., Titov, I., Welling, M., 2018. Modeling relational data with graph convolutional networks. In: The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings, Vol. 15. pp. 593–607. [http://dx.doi.org/10.1007/978-3-319-93417-4_38](https://doi.org/10.1007/978-3-319-93417-4_38).
- Schrod, P.A., 2012. Cameo: Conflict and mediation event observations event and actor codebook. Pennsylvania State Univ. 610, 35.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R., 2014. Dropout: A simple way to prevent neural networks from overfitting. J. Mach. Learn. Res. 15, 1929–1958.
- Sun, Z., Deng, Z., Nie, J., Tang, J., 2019. RotatE: Knowledge graph embedding by relational rotation in complex space. In: International Conference on Learning Representations.
- Sun, H., Zhong, J., Ma, Y., Han, Z., He, K., 2021. TimeTraveler: Reinforcement learning for temporal knowledge graph forecasting. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 8306–8319. [http://dx.doi.org/10.18653/v1/2021.emnlp-main.655](https://doi.org/10.18653/v1/2021.emnlp-main.655).
- Trivedi, R., Dai, H., Wang, Y., Song, L., 2017. Know-evolve: Deep temporal reasoning for dynamic knowledge graphs. In: International Conference on Machine Learning. pp. 3462–3471.
- Trouillon, T., Welbl, J., Riedel, S., Gaussier, É., Bouchard, G., 2016. Complex embeddings for simple link prediction. In: International Conference on Machine Learning. pp. 2071–2080.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, A., Polosukhin, I., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2017. Graph attention networks. In: International Conference on Learning Representations.
- Wang, Z., Du, H., Yao, Q., Li, X., 2022. Search to pass messages for temporal knowledge graph completion. In: Findings of the Association for Computational Linguistics. EMNLP, Abu Dhabi, United Arab Emirates, December, 7–11 2022, pp. 6160–6172.
- Wang, Q., Mao, Z., Wang, B., Guo, L., 2017. Knowledge graph embedding: A survey of approaches and applications. IEEE T. Knowl. Data En. 29, 2724–2743. [http://dx.doi.org/10.1109/TKDE.2017.2754499](https://doi.org/10.1109/TKDE.2017.2754499).
- Wu, J., Cao, M., Cheung, J.C.K., Hamilton, W.L., 2020. TeMP: Temporal message passing for temporal knowledge graph completion. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. EMNLP, pp. 5730–5746. [http://dx.doi.org/10.18653/v1/2020.emnlp-main.462](https://doi.org/10.18653/v1/2020.emnlp-main.462).
- Xie, R., Liu, Z., Jia, J., Luan, H., Sun, M., 2016a. Representation learning of knowledge graphs with entity descriptions. In: Proceedings of the AAAI Conference on Artificial Intelligence. [http://dx.doi.org/10.1609/aaai.v30i1.10329](https://doi.org/10.1609/aaai.v30i1.10329).
- Xie, R., Liu, Z., Sun, M., 2016b. Representation learning of knowledge graphs with hierarchical types. In: IJCAI. pp. 2965–2971.

- Xu, C., Nayyeri, M., Alkhoury, F., Yazdi, H., Lehmann, J., 2020. Temporal knowledge graph completion based on time series Gaussian embedding. In: The Semantic Web ISWC 2020: 19th International Semantic Web Conference, Athens, Greece, November 2–6, 2020, Proceedings, Part I Vol. 19. pp. 654–671. http://dx.doi.org/10.1007/978-3-030-62419-4_37.
- Yan, C., Zhao, F., Jin, H., 2022. ExKGR: Explainable multi-hop reasoning for evolving knowledge graph. In: Database Systems for Advanced Applications: 27th International Conference, DASFAA 2022, Virtual Event, April 11–14, 2022, Proceedings, Part I. pp. 153–161. http://dx.doi.org/10.1007/978-3-031-00123-9_11.
- Yang, B., Yih, S.W., He, X., Gao, J., Deng, L., 2015. Embedding entities and relations for learning and inference in knowledge bases. In: Proceedings of the International Conference on Learning Representations. ICLR 2015.
- Zhang, J., Shi, X., Xie, J., Ma, H., King, I., Yeung, D.Y., 2018. GaAN: Gated attention networks for learning on large and spatiotemporal graphs. In: 34th Conference on Uncertainty in Artificial Intelligence 2018. UAI 2018.
- Zhang, Z., Zhuang, F., Zhu, H., Shi, Z., Xiong, H., He, Q., 2020. Relational graph neural network with hierarchical attention for knowledge graph completion. In: Proceedings of the AAAI Conference on Artificial Intelligence. pp. 9612–9619. <http://dx.doi.org/10.1609/aaai.v34i05.6508>.
- Zhu, C., Chen, M., Fan, C., Cheng, G., Zhang, Y., 2021. Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks. In: Proceedings of the AAAI Conference on Artificial Intelligence. pp. 4732–4740. <http://dx.doi.org/10.1609/aaai.v35i5.16604>.